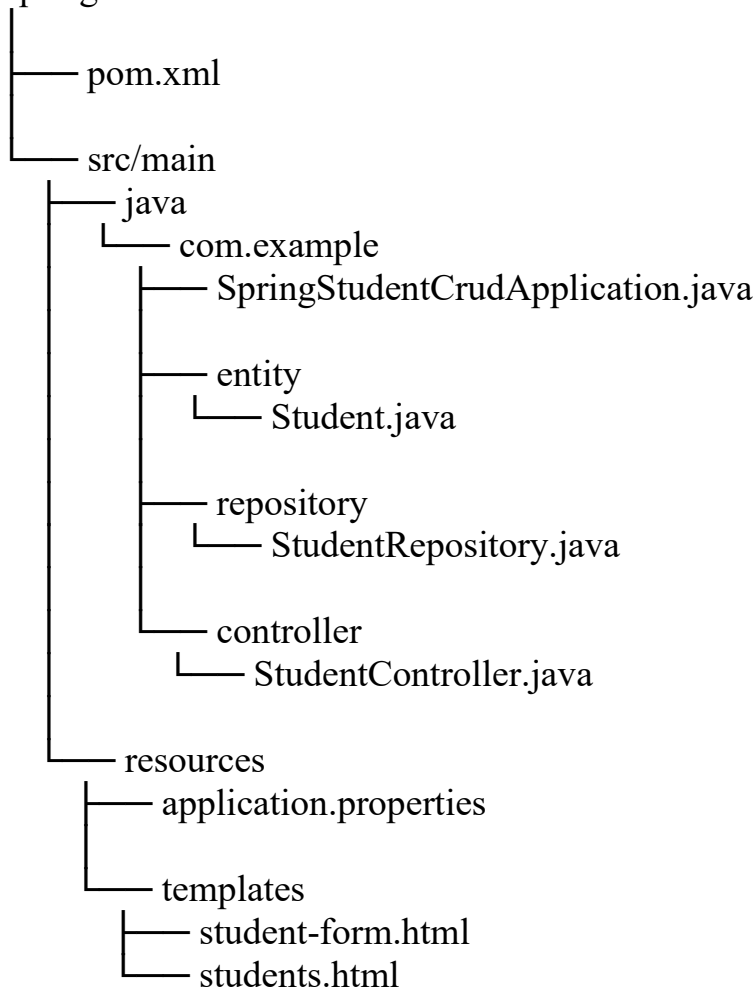
Unit : 5 PROGRAMS

- Develop an application using Spring Boot and Spring Data JPA to perform CRUD operations on a Student table without. Fields include rollno, name, course, and semester. Spring Boot simplifies the project setup, and JPA handles all database actions.

Create MySQL Database

Project Structure

SpringStudentCRUD



pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"

    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-parent</artifactId>
   <version>3.4.1</version>
   <relativePath/>
</parent>
<groupId>com.example</groupId>
<artifactId>SpringStudentCRUD</artifactId>
<version>1.0</version>
<properties>
   <java.version>17</java.version>
</properties>
<dependencies>
   <!-- Spring Web -->
   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
   </dependency>
   <!-- Spring Data JPA -->
   <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <!-- MySQL -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <!-- Thymeleaf -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Create Student.java

```
package com.example.entity;

import jakarta.persistence.Entity;

import jakarta.persistence.Id;

import jakarta.persistence.Table;

@Entity

@Table(name = "student")

public class Student {

   @Id

   private int rollno;

   private String name;

   private String course;

   private int semester;

   // Default constructor

   public Student() {

   }

   // Parameterized constructor

   public Student(int rollno, String name,

                  String course, int semester) {

      this.rollno = rollno;

      this.name = name;

      this.course = course;

      this.semester = semester;

   }
```

```
// Getters and Setters
public int getRollno() {
    return rollno;
}
public void setRollno(int rollno) {
    this.rollno = rollno;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getCourse() {
    return course;
}
public void setCourse(String course) {
    this.course = course;
}
public int getSemester() {
    return semester;
}
public void setSemester(int semester) {
    this.semester = semester;
```

```
    }
}
```

Create StudentRepository.java

```
package com.example.repository;

import com.example.entity.Student;

import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository

    extends JpaRepository<Student, Integer> {

}
```

Create StudentController.java

```
package com.example.controller;

import com.example.entity.Student;

import com.example.repository.StudentRepository;

import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.*;

@Controller

public class StudentController {

   private final StudentRepository repository;

   // Constructor

   public StudentController(StudentRepository repository) {

      this.repository = repository;

   }
```

```
@GetMapping("/")
public String showForm(Model model) {
    model.addAttribute("student", new Student());
    return "student-form";
}
@PostMapping("/save")
public String saveStudent(Student student) {
    repository.save(student);
    return "redirect:/students";
}
@GetMapping("/students")
public String showStudents(Model model) {
    model.addAttribute(
            "students",
            repository.findAll()
    );
    return "students";
}
@GetMapping("/edit/{rollno}")
public String editStudent(
        @PathVariable int rollno,
        Model model) {
    Student student =
            repository.findById(rollno).orElse(null);
```

```
        model.addAttribute("student", student);
        return "student-form";
    }
    @GetMapping("/delete/{rollno}")
    public String deleteStudent(
        @PathVariable int rollno) {
        repository.deleteById(rollno);
        return "redirect:/students";
    }
}
```

src/main/resources/application.properties

```
# MySQL Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/college
spring.datasource.username=root
spring.datasource.password=root
# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
# Server Port
server.port=8080
```

student-form.html

```
<!DOCTYPE html>
```

```
<html>
<head>
    <title>Student Form</title>
</head>
<body>
    <h2>Student Registration</h2>
    <form action="/save" method="post">
        Roll No:
        <input type="number"
               name="rollno"
               th:value="${student.rollno}"
               required>
        <br><br>
        Name:
        <input type="text"
               name="name"
               th:value="${student.name}"
               required>
        <br><br>
        Course:
        <input type="text"
               name="course"
               th:value="${student.course}"
               required>
```

```
            <br><br>
            Semester:
            <input type="number"
                name="semester"
                th:value="${student.semester}"
                required>
            <br><br>
            <input type="submit" value="Save Student">
        </form>    <br>
        <a href="/students">View All Students</a>
    </body>
    </html>
```

students.html

```
    <!DOCTYPE html>
    <html>
    <head>
        <title>Student List</title>
    </head>
    <body>
        <h2>Student List</h2>
        <table border="1" cellpadding="10">
            <tr>
                <th>Roll No</th>
```

```
            <th>Name</th>
            <th>Course</th>
            <th>Semester</th>
            <th>Action</th>
        </tr>
        <tr th:each="student : ${students}">
            <td th:text="${student.rollno}"></td>
            <td th:text="${student.name}"></td>
            <td th:text="${student.course}"></td>
            <td th:text="${student.semester}"></td>
            <td>
                <a th:href="@{/edit/{rollno}(rollno=${student.rollno})}">
                    Edit
                </a>
                <a th:href="@{/delete/{rollno}(rollno=${student.rollno})}">
                    Delete
                </a>
            </td>
        </tr>
    </table>
    <br>
    <a href="/">Add New Student</a>
</body>
</html>
```

Main Spring Boot Class

```
package com.example;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class SpringStudentCrudApplication {

   public static void main(String[] args) {

      SpringApplication.run(

            SpringStudentCrudApplication.class,

            args

      );

   }

}
```